Instituto INFNET
Escola Superior da Tecnologia da Informação
EDC - Graduação em Engenharia de Computação
EST - Fundamentos da Computação

Renato Noronha Hack

Teste de Performance - TP1

Fundamentos de Verilog e FPGA

Vitor Amadeu Souza

Rio de Janeiro, 21 de fevereiro de 2026

Padronização adotada

- Operadores lógicos:
  - AND -> *
  - OR -> +
  - NOT -> barra superior (overline) ou símbolo NOT
- Variáveis booleanas: A, B, C
- Domínio lógico: 0 e 1
- Ordem das linhas da tabela verdade: contagem binária crescente das entradas
- Regra de legibilidade: usar espaço entre operadores e operandos

Exercício 1 - FPGA x Microcontrolador x ASIC

Tabela comparativa

Critério	FPGA	Microcontrolador	ASIC
Flexibilidade	Alta: lógica digital customizável em hardware	Média: software flexível, hardware interno fixo	Baixa: função definida no projeto do chip
Custo em baixas quantidades	Médio/alto por unidade, sem custo NRE elevado	Baixo por unidade, normalmente mais barato em pequeno volume	Muito alto (alto NRE, máscaras e fabricação)
Reconfiguração	Sim, reprogramável em campo várias vezes	Parcial: firmware regravável, mas arquitetura de hardware não muda	Não (na prática, após fabricação)

Tecnologia mais adequada para prototipação

A **FPGA** é a mais adequada para prototipação de sistemas digitais, porque combina:

- alta flexibilidade lógica;
- reconfiguração rápida durante testes;
- baixo risco técnico para iterações, sem custo inicial de fabricação de chip.

Em prototipação, essa combinação normalmente é mais importante que o custo unitário.

Exercício 2 - Função F(A, B) = A AND B

Expressão booleana (notação padrão)

F(A, B) = A * B

Tabela verdade

A	B	F = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Significado lógico da operação AND

A operação AND só produz saída lógica 1 quando **todas** as entradas envolvidas estão em 1.
Em qualquer outro caso, a saída é 0.

Exercício 3 - Função F(A, B, C) = (A AND B) OR C

Expressão booleana equivalente

F = (A * B) + C

Tabela verdade

A	B	C	F = (A AND B) OR C
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Verificação de consistência

A tabela verdade é consistente com a expressão:

- se C = 1, então F = 1 independentemente de A e B;
- se C = 0, então F depende de A * B, sendo 1 apenas quando A = 1 e B = 1.

Exercício 4 - Combinação de expressões booleanas

Expressões dadas

- E1 = A * B
- E2 = NOT C

Relação com E1 e E2

F(A, B, C) = E1 + E2

Função construída (em termos de A, B e C)

F(A, B, C) = (A * B) + (NOT C)

Condições para F = 1

A saída F é 1 quando ocorre pelo menos uma das condições:

- C = 0 (pois NOT C = 1);
- A = 1 e B = 1 (pois A AND B = 1), mesmo com C = 1.

Forma resumida:
F = 1 <=> (overline(C) = 1) ou (A * B = 1)

Exercício 5 - Análise de F(A, B, C) = (A OR B) AND (NOT C)

Expressão booleana F = (A + B) * overline(C)

Comportamento lógico

A saída F vale 1 somente quando:

- C = 0; e
- pelo menos uma entre A ou B vale 1.

Se C = 1, a saída sempre é 0 por causa do fator NOT C.

Tabela verdade

A	B	C	F = (A OR B) AND (NOT C)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Coerência entre descrição e tabela

A tabela confirma a descrição textual: os únicos casos com F = 1 são:

(A, B, C) = (0, 1, 0), (1, 0, 0), (1, 1, 0).

Exercício 6 - Interpretação por LUT

Função `F(A, B, C) = (A OR B) AND C`

Tabela verdade

A	B	C	F = (A OR B) AND C
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Interpretação como conteúdo de LUT

Uma LUT implementa função booleana mapeando combinações de entrada para bits de saída.
Para `A, B, C`, existem `2^3 = 8` combinações, então a LUT armazena 8 bits.

Se o endereço da LUT seguir a ordem binária `ABC = 000` até `ABC = 111`, o conteúdo é:
`[0, 0, 0, 1, 0, 1, 0, 1]`

Esse vetor é exatamente a tabela verdade da função.

Número de entradas da LUT

São necessárias **3 entradas** (LUT de 3 variáveis).

Exercício 7 - Comparação entre F1 e F2

Funções

- F1(A, B) = A AND B
- F2(A, B, C) = (A AND B) OR C

Tabela verdade de F1

A	B	F1 = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Tabela verdade de F2

A	B	C	F2 = (A AND B) OR C
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1

1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Entradas de LUT necessárias

- F1: LUT de **2 entradas** (2^2 = 4 posições).
- F2: LUT de **3 entradas** (2^3 = 8 posições).

Impacto do aumento do número de variáveis

Quando o número de variáveis aumenta, o número de combinações cresce exponencialmente (2^n).
Isso aumenta:

- o tamanho da tabela verdade;
- a quantidade de informação armazenada em LUT;
- a complexidade de análise e validação lógica.

Exercício 8 - Função F(A, B, C) = A OR (B AND C)

Expressão booleana F = A + (B * C)

Tabela verdade

A	B	C	F = A OR (B AND C)
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Comportamento lógico (descrição técnica)

A saída F é 1 quando:

- A = 1 (independente de B e C), ou
- B = 1 e C = 1 simultaneamente.

Logo, com A = 0, a função se reduz a B AND C.

Exercício 9 - Flexibilidade lógica das tecnologias

Comparação

Tecnologia	Flexibilidade lógica	Relação com reconfiguração
FPGA	Alta	Lógica interna pode ser reconfigurada em campo para novas funções digitais
Microcontrolador	Média	Reprograma firmware, mas não altera a estrutura de lógica de hardware interna
ASIC	Baixa	Função lógica fica fixa após fabricação

Tecnologia com maior flexibilidade conceitual

A **FPGA** oferece maior flexibilidade conceitual para sistemas digitais genéricos, porque permite:

- redefinir a função lógica em nível de hardware;
- testar arquiteturas diferentes no mesmo dispositivo;
- adaptar o sistema sem refabricar chip.

Microcontrolador é flexível no software, mas limitado para lógica digital customizada em paralelo. ASIC é o menos flexível após produzido.

Exercício 10 - Justificativa de modelagem booleana

Função F(A, B, C) = (A AND B) OR (NOT C)

Por que a função é descrita integralmente por álgebra booleana e tabela verdade

A expressão booleana define formalmente a relação lógica entre entradas e saída.
A tabela verdade enumera todas as combinações possíveis (2^3 = 8) e o valor de F em cada caso.

Como a lógica digital é binária e finita por combinação de entradas, expressão + tabela verdade fornecem uma especificação completa e sem ambiguidades do comportamento da função.

Importância antes da implementação em hardware

Essa representação prévia é essencial para:

- validar o comportamento esperado antes de sintetizar ou montar circuito;
- evitar erros conceituais que geram retrabalho em FPGA, microcontrolador ou ASIC;
- facilitar revisão técnica, documentação e testes de verificação.

Em resumo, primeiro se valida a lógica; depois se escolhe a tecnologia de implementação.

Exercício 11 - Padronização e organização documental

Este item foi atendido neste relatório por:

- padronização única de notação booleana;
- revisão e organização de todas as tabelas verdade;
- consolidação lógica e sequencial das análises técnicas.

Exercício 12 - Validação final

Função revisada `F(A, B, C) = (A OR B) AND (NOT C)`

Expressão booleana final validada

`F = (A + B) * overline(C)`

Tabela verdade final

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Confirmação de equivalência

A expressão e a tabela representam exatamente o mesmo comportamento:

- `F = 1` somente quando `C = 0` e ao menos uma entre `A` ou `B` é `1`;

- nos demais casos, $F = 0$.

Versão final revisada e validada registrada neste arquivo.